CS316

INTRODUCTION TO AI AND DATA SCIENCE

Topic 7.2

Dimensionality reduction
Introduction to PCA

Instructor: Dr. Heyfa Ammar

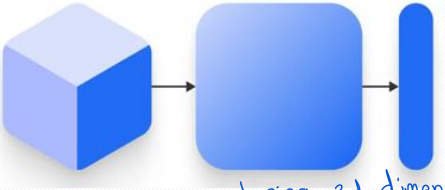
Dimensionality reduction : Principal Component Analysis (PCA)

- What is dimensionality reduction and why?
- Steps required for dimensionality reduction using PCA
- Mathematical background on eigenvalues and eigenvectors
- Eigen decomposition
- Selection of the principal components
- PCA transformation

Dimensionality reduction

finding redundant features and combining them into one feature by extracting new features from the original features

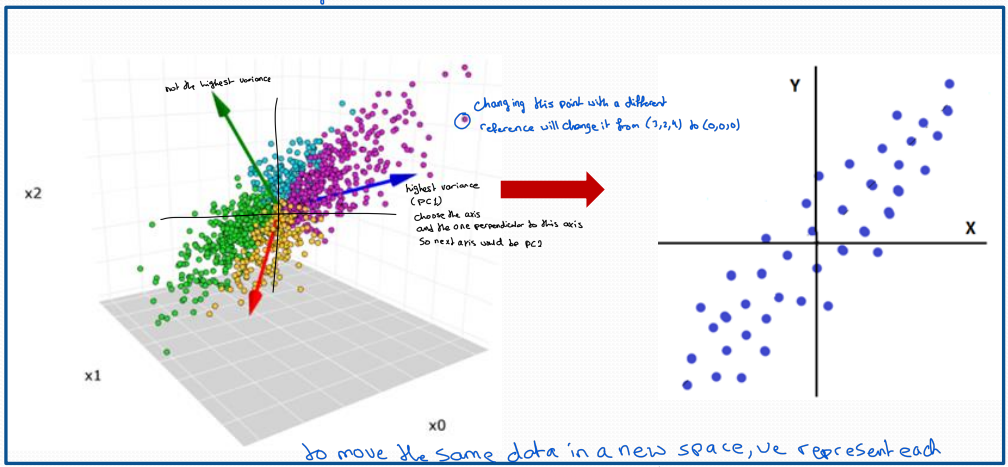
reding features but keeping the most imp info abt the features

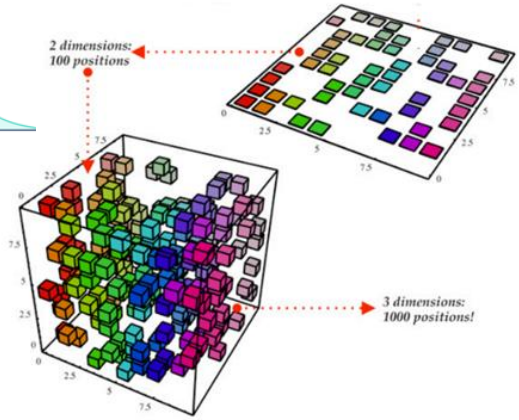






reducing 3d dimensions, u need to know the most relevant features

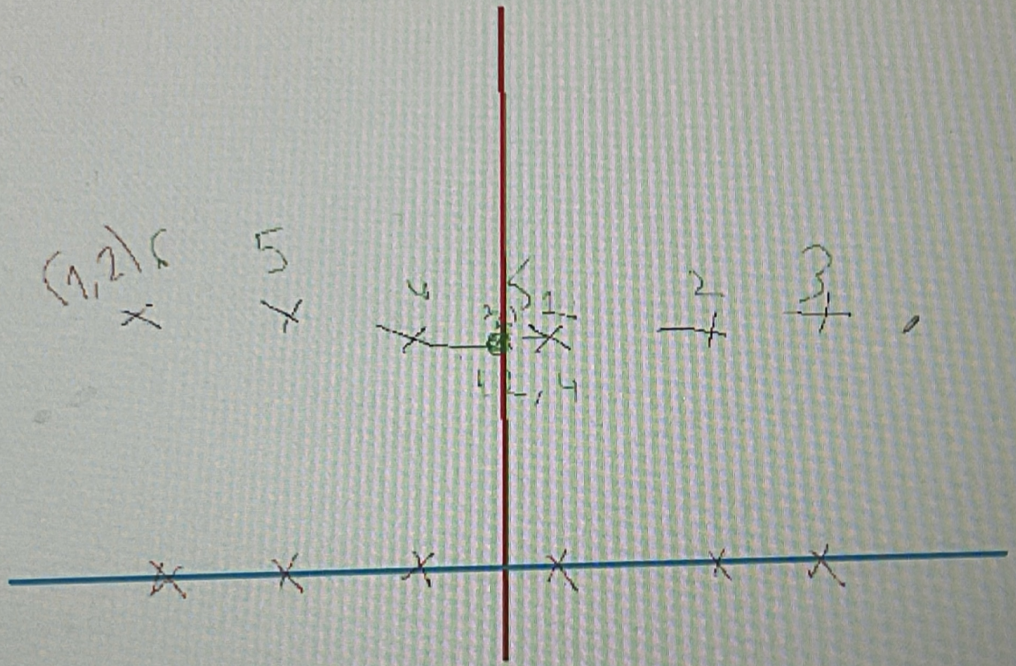




How will we choose the best axis that makes us lose the least amount of information?

We will use PCA

to move the same data in a new space, we represent each space by an axis, changing the reference changes the space



We want to reduce the set of features from 2d (x's) and 1d using either the blue line or red line

The blue line keeps most of the info abt the points

Motivating Example – Why Use PCA?

- Imagine you're analyzing a dataset of students' performance across multiple subjects (Math, Physics, Chemistry, etc.).
- **Challenge**: The dataset has many variables, and we suspect that some subjects are correlated (e.g., students who do well in Math may also do well in Physics).
- **Goal**: Simplify the dataset by reducing the number of subjects, while retaining the most important information about student performance.

Dataset:

- 10 students with scores in 4 subjects.
- Our goal is to simplify the analysis of student performance by reducing the number of features (subjects) while keeping the most important information.

Student	Math	Physics	Chemistry	English
1	85	82	88	79
2	78	75	79	74
3	92	90	94	85
...	...	...	...	...
10	87	86	89	82

```
data = {
    'Math': [85, 78, 92, 88, 76, 95, 89, 84, 91, 87],
    'Physics': [82, 75, 90, 85, 73, 94, 88, 81, 89, 86],
    'Chemistry': [88, 79, 94, 91, 77, 97, 90, 86, 92, 89],
    'English': [79, 74, 85, 80, 70, 88, 83, 77, 84, 82]
}
```

One technique to reduce dimensionality is the Principal Component Analysis (PCA)

Step 1 – Standardize the Data (Normalization)

Equation:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X is the original data.
- μ is the mean.
- σ is the standard deviation.

```
# Convert the data into a DataFrame
df = pd.DataFrame(data)

# Step 1: Standardize the data
# Subtract the mean and divide by standard deviation to standardize
df_standardized = (df - df.mean()) / df.std()
```

Numerical Example (Math):

$$Z_{\text{Math, Student 1}} = \frac{85 - 87.7}{5.49} = -0.49$$

This step ensures that each subject contributes equally to the analysis.

Step 1 – Standardize the Data (Normalization)

Student	Math	Physics	Chemistry	English
1	-0.25	-0.35	-0.05	-0.22
2	-1.42	-1.40	-1.49	-1.15
3	0.92	0.86	0.91	0.89
4	0.25	0.11	0.43	-0.04
5	-1.75	-1.70	-1.81	-1.88
6	1.42	1.46	1.39	1.44
7	0.42	0.56	0.27	0.52
8	-0.42	-0.50	-0.37	-0.59
9	0.75	0.71	0.59	0.70
10	0.08	0.26	0.11	0.33

Step 2 – Calculate the Covariance Matrix

What is the Covariance Matrix?

- It shows how much each feature (subject) varies with others.

Covariance Matrix:

$$\text{Cov} = \begin{bmatrix} \text{Var(Math)} & \text{Cov(Math, Physics)} & \dots \\ \text{Cov(Physics, Math)} & \text{Var(Physics)} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Example:

- Math and Physics might have high covariance because they are closely related.

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})(Y_i - \bar{Y})$$

n is the number of samples in the dataset
$\bar{X}$, $\bar{Y}$ are the mean values of respectively X and Y

Step 2 – Calculate the Covariance Matrix

```python
### Step 2: Calculate the Covariance Matrix
cov_matrix = np.cov(df_standardized.T)
print("\nCovariance Matrix:\n", cov_matrix)
```

```
Covariance Matrix:
 [[1.         0.99465467 0.99265481 0.98433819]
 [0.99465467 1.         0.98031195 0.99155631]
 [0.99265481 0.98031195 1.         0.96962717]
 [0.98433819 0.99155631 0.96962717 1.        ]]
```

$$\text{Covariance Matrix} = \begin{bmatrix} 1.00 & 0.99 & 0.99 & 0.98 \\ 0.99 & 1.00 & 0.98 & 0.99 \\ 0.99 & 0.98 & 1.00 & 0.97 \\ 0.98 & 0.99 & 0.97 & 1.00 \end{bmatrix}$$

How can we exploit this matrix to reduce the number of features ?

Mathematical background on EigenValues and EigenVectors

Eigenvalues and Eigenvectors Explained Simply

Imagine a Transformation:

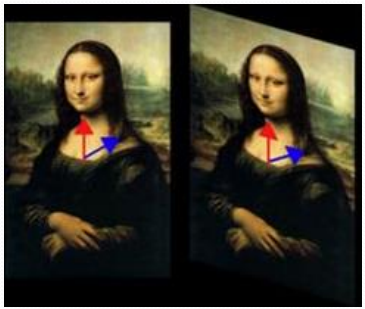
- Think of a transformation like stretching, squeezing, or rotating objects in space.
- In math, we represent these transformations using **matrices** that act on **vectors**.

Special Vectors That Keep Their Direction:

- When we apply a transformation to most vectors, they change direction and length.
- However, **eigenvectors** are special because they **do not change direction** when transformed.
- They may get stretched (longer), compressed (shorter), or flipped (direction reversed), but they still point along the same line.

Scaling Factors—Eigenvalues:

- The amount by which an eigenvector is stretched or compressed is called an **eigenvalue**.
- If the eigenvalue is:
  - **Greater than 1**: The eigenvector is stretched.
  - **Between 0 and 1**: The eigenvector is compressed.
  - **Negative**: The eigenvector flips direction.
  - **Zero**: The eigenvector collapses to a point (uncommon in practical scenarios).

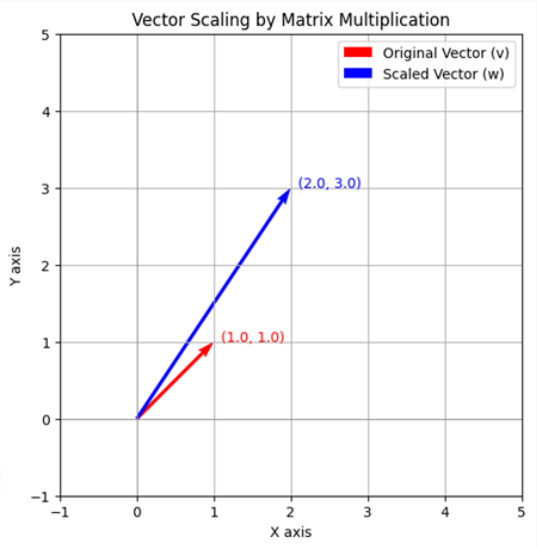


Scaling Vectors with Matrices

3. **Numerical Example:**
   - **Given:**
     - Vector $v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
     - Scaling factors $s_x = 2$ and $s_y = 3$
   - **Calculation:**
     - Applying S to v:

$$w = Sv = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

     - The vector v is scaled to w, changing its length but not its orientation relative to the axes.

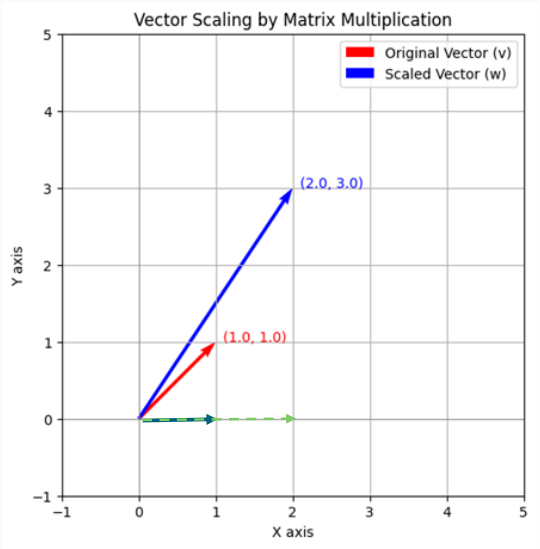


Scaling Vectors with Matrices

- **Given:**
  - Vector $v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
  - Scaling factors $s_x = 2$ and $s_y = 3$
- **Calculation:**
  - Applying S to v:

$$w = Sv = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

- **Interpretation:**
  - The vector v is scaled to w, changing its length but maintaining its direction along the x-axis.

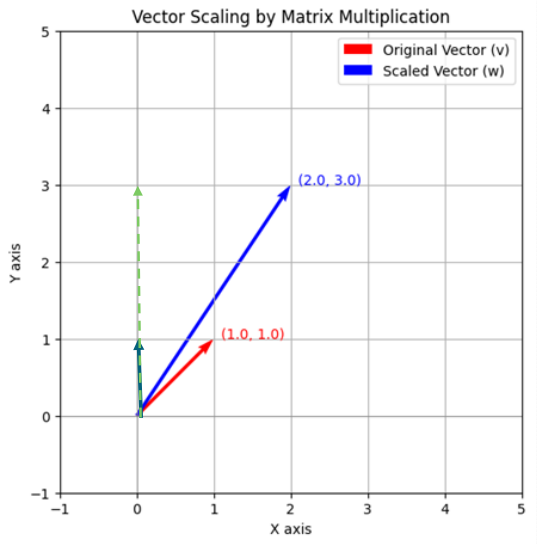


Scaling Vectors with Matrices

- **Given**:
  - Vector $v = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
  - Scaling factors $s_x = 2$ and $s_y = 3$
- **Calculation**:
  - Applying S to v:

$$w = Sv = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

- **Interpretation**:
  - The vector v is scaled to w, changing its length along the y-axis but maintaining its direction along the y-axis.



This is one of the properties of the eigenvectors

Eigenvectors and Eigenvalues in Matrix Scaling

Eigenvectors:

- Vectors that remain in the same direction after a matrix transformation.
- In the scaling matrix:

$$S = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

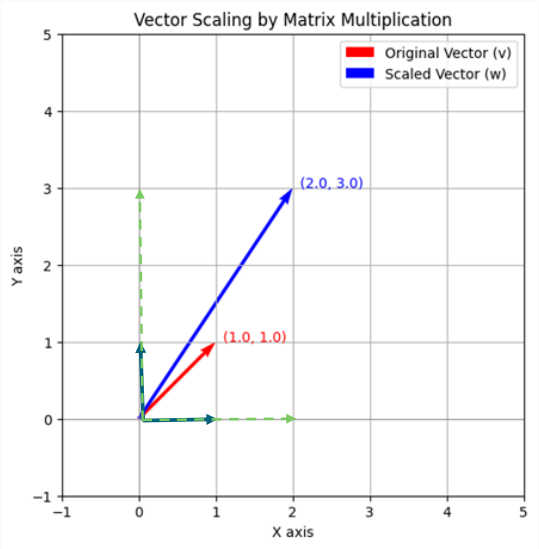
  The vectors $[1, 0]$ and $[0, 1]$ are eigenvectors.

Eigenvalues:

- Scalars by which the eigenvectors are scaled.
- For the scaling matrix S:
  - Eigenvector $v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ has eigenvalue $\lambda_1 = 2$.
  - Eigenvector $v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ has eigenvalue $\lambda_2 = 3$.

Conclusion:

- The matrix S scales the eigenvector $[1, 0]$ by 2 and $[0, 1]$ by 3.
- These eigenvectors retain their original direction, and the eigenvalues represent the factors by which they are stretched.



Impact of the Largest Eigenvalue in Transformations

Key Concept:

- **Dominant Eigenvalue**: The eigenvalue with the highest magnitude has the greatest effect on scaling the corresponding eigenvector. λ gives info about the variance
  High eigen vector → high variance (most contributed λ)
- In the matrix:

$$S = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

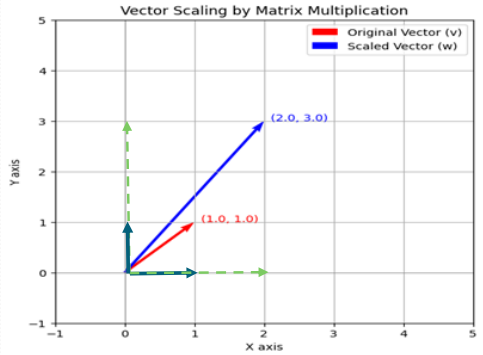
  - Eigenvalue $\lambda_2 = 3$ (corresponding to $v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$) is the largest.
  - This means the scaling along the y-axis has a **greater impact** than scaling along the x-axis.

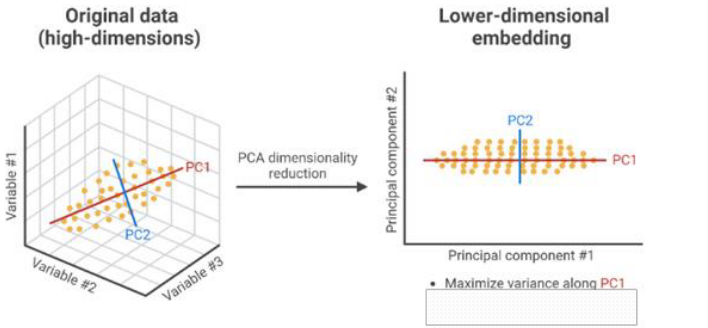
Application in Data Science:

- In **Principal Component Analysis (PCA)**:
  - The direction associated with the largest eigenvalue (principal component) captures the most variance in the data.
  - The principal components with larger eigenvalues have a **greater influence** on the dataset's structure, reducing dimensionality while retaining the most significant information.

Conclusion:

- The largest eigenvalue not only represents the greatest scaling effect but also plays a key role in identifying key patterns and structures in data.





Step 3 – Eigen Decomposition

- **Eigenvalues**:

$$\text{Eigenvalues} = [3.9566, 0.0338, 0.0079, 0.0016]$$

- **Eigenvectors**:

$$\begin{bmatrix} 0.5019 & 0.2018 & -0.3465 & -0.7664 \\ 0.5013 & -0.2614 & -0.6227 & 0.5409 \\ 0.4982 & 0.6969 & 0.3959 & 0.3308 \\ 0.4986 & -0.6366 & 0.5792 & -0.1029 \end{bmatrix}$$

Interpretation:

- The first eigenvalue (**3.9566**) explains the largest portion of the variance, and the first eigenvector points in the direction of maximum variation.

```
### Step 3: Compute Eigenvalues and Eigenvectors of the Covariance Matrix
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("\nEigenvalues:\n", eigenvalues)
print("\nEigenvectors:\n", eigenvectors)
```

```
Eigenvalues:
 [3.95661230e+00 3.38356070e-02 1.65383113e-03 7.89826523e-03]

Eigenvectors:
 [[ 0.50190456  0.20177656 -0.76638646 -0.34645321]
 [ 0.50125904 -0.26142512  0.54094842 -0.62271268]
 [ 0.49822683  0.69686993  0.33080625  0.39586558]
 [ 0.49859925 -0.63664388 -0.1029263   0.57921458]]
```

Question

Original data

Math	Physics	Chemistry	English
85	82	88	79
78	75	79	74
92	90	94	85
...	...	...	...
87	86	89	82

Covariance matrix

$$\begin{bmatrix} 1.00 & 0.99 & 0.99 & 0.98 \\ 0.99 & 1.00 & 0.98 & 0.99 \\ 0.99 & 0.98 & 1.00 & 0.97 \\ 0.98 & 0.99 & 0.97 & 1.00 \end{bmatrix}$$

- **Eigenvalues:**

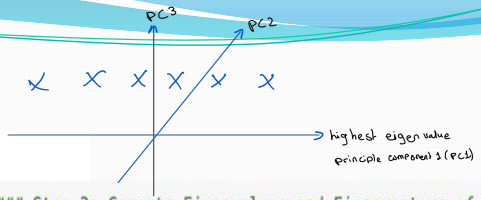
$$[3.9566, 0.0338, 0.0079, 0.0016]$$

- **Eigenvectors:**

$$\begin{bmatrix} 0.5019 & 0.2018 & -0.3465 & -0.7664 \\ 0.5013 & -0.2614 & -0.6227 & 0.5409 \\ 0.4982 & 0.6969 & 0.3959 & 0.3308 \\ 0.4986 & -0.6366 & 0.5792 & -0.1029 \end{bmatrix}$$

Step 3 – Eigen Decomposition

Ex: first eigenvalue has 60% and 2nd has 20%
If we want 80% then we'll only have 2 features
rather than having all features while not losing much data



- **Eigenvalues:**

$$\text{Eigenvalues} = [3.9566, 0.0338, 0.0079, 0.0016]$$

most contribution

- **Eigenvectors:**

$$\begin{bmatrix} 0.5019 & 0.2018 & -0.3465 & -0.7664 \\ 0.5013 & -0.2614 & -0.6227 & 0.5409 \\ 0.4982 & 0.6969 & 0.3959 & 0.3308 \\ 0.4986 & -0.6366 & 0.5792 & -0.1029 \end{bmatrix}$$

```
### Step 3: Compute Eigenvalues and Eigenvectors of the Covariance Matrix
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("\nEigenvalues:\n", eigenvalues)
print("\nEigenvectors:\n", eigenvectors)
```

Interpretation:

- The first eigenvalue (**3.9566**) explains the largest portion of the variance, and the first eigenvector points in the direction of maximum variation.

```
Eigenvalues:
 [3.95661230e+00 3.38356070e-02 1.65383113e-03 7.89826523e-03]

Eigenvectors:
 [[ 0.50190456  0.20177656 -0.76638646 -0.34645321]
 [ 0.50125904 -0.26142512  0.54094842 -0.62271268]
 [ 0.49822683  0.69686993  0.33080625  0.39586558]
 [ 0.49859925 -0.63664388 -0.1029263   0.57921458]]
```

Step4 - Sorting Eigenvalues and Eigenvectors

Calculate → decompress eigen vectors → sort eigenvectors/ eigen values

Goal: Arrange the eigenvalues and their corresponding eigenvectors in descending order of importance (variance explained).

Numerical Results: Dominant EigenVector and EigenValue

- **Eigenvalues** (sorted in descending order):

$$\lambda_1 = 3.9566, \quad \lambda_2 = 0.0338, \quad \lambda_3 = 0.0079, \quad \lambda_4 = 0.0016$$

most important

- **Eigenvectors** (sorted based on the eigenvalues):

$$\text{Eigenvector}_1 = \begin{bmatrix} 0.5019 \\ 0.5013 \\ 0.4982 \\ 0.4986 \end{bmatrix}, \quad \text{Eigenvector}_2 = \begin{bmatrix} 0.2018 \\ -0.2614 \\ 0.6969 \\ -0.6366 \end{bmatrix}, \ldots$$

Interpretation:

- The first eigenvalue ($\lambda_1 = 3.9566$) explains the majority of the variance in the data.
- Eigenvector 1 is the direction that captures the most variation in the data.

```
### Step 4: Sort Eigenvalues and Eigenvectors (i
# Sort the eigenvalues and their corresponding e
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]
```

```
Eigenvalues:
 [3.95661230e+00 3.38356070e-02 1.65383113e-03 7.89826523e-03]

Eigenvectors:
 [[ 0.50190456  0.20177656 -0.76638646 -0.34645321]
 [ 0.50125904 -0.26142512  0.54094842 -0.62271268]
 [ 0.49822683  0.69686993  0.33080625  0.39586558]
 [ 0.49859925 -0.63664388 -0.1029263   0.57921458]]
```

Step4 – Sorting Eigenvalues and Eigenvectors

Goal: Arrange the eigenvalues and their corresponding eigenvectors in descending order of importance (variance explained).

Numerical Results:

- **Eigenvalues** (sorted in descending order):

$$\lambda_1 = 3.9566, \quad \lambda_2 = 0.0338, \quad \lambda_3 = 0.0079, \quad \lambda_4 = 0.0016$$

- **Eigenvectors** (sorted based on the eigenvalues):

$$\text{Eigenvector}_1 = \begin{bmatrix} 0.5019 \\ 0.5013 \\ 0.4982 \\ 0.4986 \end{bmatrix}, \quad \text{Eigenvector}_2 = \begin{bmatrix} 0.2018 \\ -0.2614 \\ 0.6969 \\ -0.6366 \end{bmatrix}, \ldots$$

Interpretation:

- The first eigenvalue ($\lambda_1 = 3.9566$) explains the majority of the variance in the data.
- Eigenvector 1 is the direction that captures the most variation in the data.

```
### Step 4: Sort Eigenvalues and Eigenvectors (i
# Sort the eigenvalues and their corresponding e
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]
```

```
Eigenvalues:
 [3.95661230e+00 3.38356070e-02 1.65383113e-03 7.89826523e-03]

Eigenvectors:
 [[ 0.50190456  0.20177656 -0.76638646 -0.34645321]
 [ 0.50125904 -0.26142512  0.54094842 -0.62271268]
 [ 0.49822683  0.69686993  0.33080625  0.39586558]
 [ 0.49859925 -0.63664388 -0.1029263   0.57921458]]
```

[1.01 , 1.6 , 3.61 , 2.5] : eigenvalues

[vect1 , vect2, vect3, vect4] : eigenvectors

sort the eigenvalues:

sorted eigenvalues => [3.61, 2.5 , 1.6 , 1.01]

sort_index=[2 , 3, 1, 0] After sorting, the indexes are different so we save the index to not change the positions

sorted_eigenvect=[vect3, vect4, vect2, vect1]

Importance of Eigenvalue Decomposition in Feature Reduction

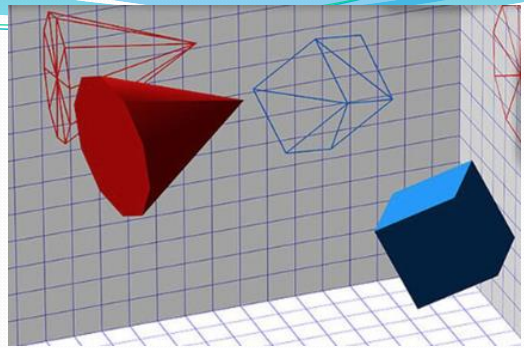
- **Simplification**: Eigenvalue decomposition helps identify the principal components (directions in the feature space that maximize variance). This process transforms correlated features into a set of linearly uncorrelated features called principal components.
- **Dimensionality Reduction**:
  - **Eigenvalues**: Indicate the amount of variance carried by each principal component. A higher eigenvalue means more variance (information).
  - **Eigenvectors**: Define the direction of each principal component in the original feature space.

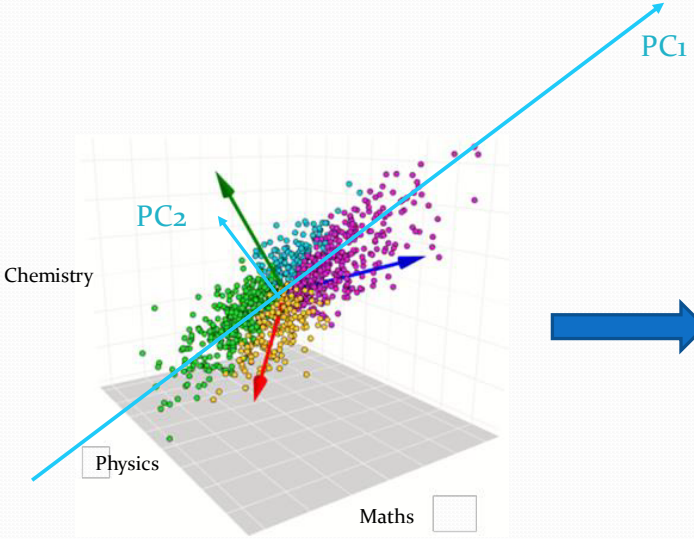
Checkpoint

What did we do so far ?
And what is next ?

Step 1& normalization
2& calculate co

Projection







PCA Transformation

Goal: Project the standardized data onto the eigenvectors (principal components).

Transformation Equation:

$$Z_{\text{PC}} = X_{\text{standardized}} \cdot \text{Eigenvector}$$

PCA Transformed Data:

Student	PC1	PC2	PC3	PC4
1	-0.434	0.148	0.155	0.011
2	-2.727	-0.227	0.113	-0.044
3	1.788	0.031	0.021	-0.029
4	0.375	0.347	-0.003	0.012
...	...	...	...	...
10	0.392	-0.184	0.048	0.077

Interpretation:

- The dataset has been transformed into the principal component space, where each student is now represented by principal components (PC1, PC2, PC3, PC4).

PCA Transformation

Goal: Project standardized data onto principal components (eigenvectors).

Compact Transformation Equation:

$$Z_{\text{PC}} = X_{\text{standardized}} \cdot \text{Eigenvector}$$

Where:

- $X_{\text{standardized}}$ is the matrix of standardized data.
- Eigenvector is the matrix of eigenvectors.
- Z_{PC} is the transformed data in principal component space.

PCA Transformation

Goal: Project standardized data onto principal components (eigenvectors).

Compact Transformation Equation:

$$Z_{\text{PC}} = X_{\text{standardized}} \cdot \text{Eigenvector}$$ (Dot product)

Where:

- $X_{\text{standardized}}$ is the matrix of standardized data.
- Eigenvector is the matrix of eigenvectors.
- Z_{PC} is the transformed data in principal component space.

Expanded Equation for PC1:

For each student (row in the dataset), the principal component Z_{PC1} is computed as:

$$Z_{\text{PC1}} = (X_1 \times e_{11}) + (X_2 \times e_{12}) + (X_3 \times e_{13}) + (X_4 \times e_{14})$$

Where:

- X_1, X_2, X_3, X_4 are the standardized values for Math, Physics, Chemistry, and English.
- $e_{11}, e_{12}, e_{13}, e_{14}$ are the elements of the first eigenvector.

Numerical Application (Student 1)

Student	Math	Physics	Chemistry	English
1	-0.25	-0.35	-0.05	-0.22

- **Standardized data for Student 1:**

$$X = \begin{bmatrix} -0.25 & -0.35 & -0.05 & -0.22 \end{bmatrix}$$

- **First Eigenvector:**

project each x to the eigen vector

$$\text{Eigenvector}_1 = \begin{bmatrix} 0.5019 \\ 0.5013 \\ 0.4982 \\ 0.4986 \end{bmatrix}$$

- **Computation of Z_{PC1}:**

$$Z_{PC1} = (-0.25 \times 0.5019) + (-0.35 \times 0.5013) + (-0.05 \times 0.4982) + (-0.22 \times 0.4986)$$

$$Z_{PC1} = -0.1255 + -0.1755 + -0.0249 + -0.1097 = -0.434$$

Numerical Application (Student 1)

Student	Math	Physics	Chemistry	English
1	-0.25	-0.35	-0.05	-0.22

Numerical Application for PC2 (Student 1):

- **Second Eigenvector:**

$$\text{Eigenvector}_2 = \begin{bmatrix} 0.2018 \\ -0.2614 \\ 0.6969 \\ -0.6366 \end{bmatrix}$$

- **Computation of Z_{PC2}:**

$$Z_{PC2} = (-0.25 \times 0.2018) + (-0.35 \times -0.2614) + (-0.05 \times 0.6969) + (-0.22 \times -0.6366)$$

$$Z_{PC2} = -0.0505 + 0.0915 + -0.0348 + 0.1401 = 0.147$$

Numerical Application (Student 1)

Transformed Data for Student 1:

- **PC1**: -0.434
- **PC2**: 0.147

Summary:

- **PC1** represents the direction with the highest variance (captured by Eigenvector 1).
- **PC2** adds more nuanced variation (captured by Eigenvector 2).
- This transformation reduces data dimensionality while retaining essential patterns.

Transforming Data Using PCA - Case of Dominant Eigenvector

Objective:

- Demonstrate how to transform high-dimensional standardized data into a new feature space using Principal Component Analysis (PCA).

Dataset and Standardization:

- **Standardized Data:**

Features: Math, Physics, Chemistry, English

$$\begin{bmatrix} -0.2506 & -0.3467 & -0.0480 & -0.2217 \\ -1.4200 & -1.4019 & -1.4869 & -1.1456 \\ 0.9188 & 0.8592 & 0.9113 & 0.8869 \\ 0.2506 & 0.1055 & 0.4317 & -0.0370 \\ -1.7541 & -1.7033 & -1.8066 & -1.8847 \\ 1.4200 & 1.4621 & 1.3909 & 1.4413 \\ 0.4176 & 0.5577 & 0.2718 & 0.5174 \\ -0.4176 & -0.4974 & -0.3677 & -0.5913 \\ 0.7517 & 0.7085 & 0.5915 & 0.7022 \\ 0.0835 & 0.2563 & 0.1119 & 0.3326 \end{bmatrix}$$

First Eigenvector (Principal Component):

- **Eigenvector 1:**

$$\begin{bmatrix} 0.5019 & 0.5013 & 0.4982 & 0.4986 \end{bmatrix}$$

Transformation Formula:

- **Projection Calculation:**

$$Z_1 = \text{Data} \cdot \text{Eigenvector}_1$$

$$Z_1 = \begin{bmatrix} -0.2506 & -0.3467 & -0.0480 & -0.2217 \\ -1.4200 & -1.4019 & -1.4869 & -1.1456 \\ \cdots & & & \\ 0.0835 & 0.2563 & 0.1119 & 0.3326 \end{bmatrix} \cdot \begin{bmatrix} 0.5019 \\ 0.5013 \\ 0.4982 \\ 0.4986 \end{bmatrix}$$

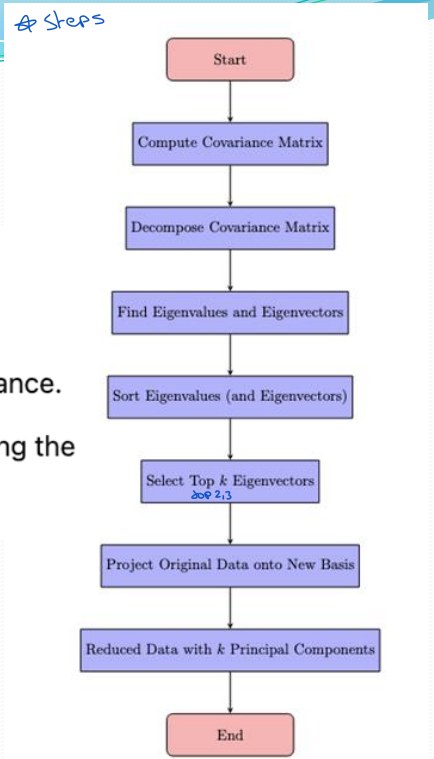
Results of Projection:

- **Projected Features (Z_1 Values):**

$$\begin{bmatrix} -0.4340 \\ -2.7274 \\ 1.7881 \\ 0.3753 \\ -3.5740 \\ 2.8572 \\ 0.8826 \\ -0.9369 \\ 1.3773 \\ 0.3920 \end{bmatrix}$$

Process of Reduction

1. **Decompose** the covariance matrix to find its eigenvalues and eigenvectors.
2. **Sort** the eigenvalues (and corresponding eigenvectors) from highest to lowest.
3. **Select** the top k eigenvectors as principal components to capture the significant variance.
4. **Project** the original data onto the space defined by the selected eigenvectors, reducing the dimensionality from four features to k principal components.



Transforming Data Using PCA

Interpretation and Benefits:

- **Dimensionality Reduction**: Reduces the dataset dimensions from four features to a single principal component.
- **Variance Capture**: Z_1 captures the most variance across the dataset, enabling efficient data representation and analysis.

Questions

- What is a projection of a vector on another vector in space? dot product of the data in the new space
- Why covariance is used to select features? reflects the dependancy of features
- Why features projected on principal components represent the original data?

we find the highest variance finding the most contributed data

Question

Is PCA Feature Representative of the Original Data?

Math	Physics	Chemistry	English
85	82	88	79
78	75	79	74
92	90	94	85
88	85	91	80
76	73	77	70
95	94	97	88
89	88	90	83
84	81	86	77
91	89	92	84
87	86	89	82



Features: Math, Physics, Chemistry, English

$$\begin{bmatrix} -0.2506 & -0.3467 & -0.0480 & -0.2217 \\ -1.4200 & -1.4019 & -1.4869 & -1.1456 \\ 0.9188 & 0.8592 & 0.9113 & 0.8869 \\ 0.2506 & 0.1055 & 0.4317 & -0.0370 \\ -1.7541 & -1.7033 & -1.8066 & -1.8847 \\ 1.4200 & 1.4621 & 1.3909 & 1.4413 \\ 0.4176 & 0.5577 & 0.2718 & 0.5174 \\ -0.4176 & -0.4974 & -0.3677 & -0.5913 \\ 0.7517 & 0.7085 & 0.5915 & 0.7022 \\ 0.0835 & 0.2563 & 0.1119 & 0.3326 \end{bmatrix}$$



$$\begin{bmatrix} -0.4340 \\ -2.7274 \\ 1.7881 \\ 0.3753 \\ -3.5740 \\ 2.8572 \\ 0.8826 \\ -0.9369 \\ 1.3773 \\ 0.3920 \end{bmatrix}$$

$$\begin{bmatrix} 1.00 & 0.99 & 0.99 & 0.98 \\ 0.99 & 1.00 & 0.98 & 0.99 \\ 0.99 & 0.98 & 1.00 & 0.97 \\ 0.98 & 0.99 & 0.97 & 1.00 \end{bmatrix}$$

Validating PCA's Representation of Original Data

Objective:

- Illustrate PCA's capability to maintain the essential characteristics of the original data through dimensionality reduction.

$$\begin{bmatrix} 1.00 & 0.99 & 0.99 & 0.98 \\ 0.99 & 1.00 & 0.98 & 0.99 \\ 0.99 & 0.98 & 1.00 & 0.97 \\ 0.98 & 0.99 & 0.97 & 1.00 \end{bmatrix}$$

Key Points:

1. **Variance as Information**:
   - PCA preserves variance, which represents the most informative aspects of the data.
2. **Principal Components**:
   - Identify directions that maximize data variance.
   - Capture core patterns and structures in the data.
3. **Mathematical Foundation**:
   - **Covariance Matrix**: Identifies the covariance between all pairs of features.
   - **Eigenvalues and Eigenvectors**: Determine the amount of variance each component holds.

Numerical Results: Dominant EigenVector and EigenValue

- **Eigenvalues** (sorted in descending order):

$$\lambda_1 = 3.9566, \quad \lambda_2 = 0.0338, \quad \lambda_3 = 0.0079, \quad \lambda_4 = 0.0016$$

- **Eigenvectors** (sorted based on the eigenvalues):

$$\text{Eigenvector}_1 = \begin{bmatrix} 0.5019 \\ 0.5013 \\ 0.4982 \\ 0.4986 \end{bmatrix}, \quad \text{Eigenvector}_2 = \begin{bmatrix} 0.2018 \\ -0.2614 \\ 0.6969 \\ -0.6366 \end{bmatrix}, \ldots$$

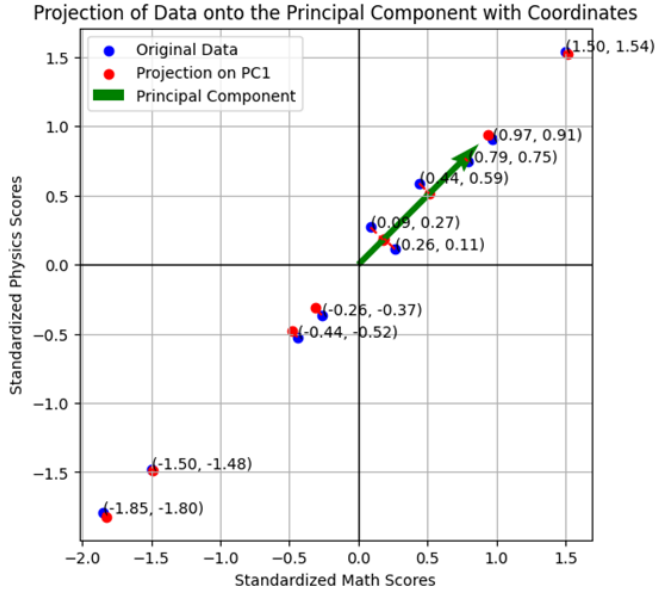
Interpretation:

- The first eigenvalue ($\lambda_1 = 3.9566$) explains the majority of the variance in the data.
- Eigenvector 1 is the direction that captures the most variation in the data.

Validating PCA's Representation of Original Data

$$Z_1 = [-1.50, -1.85, 0.97, 0.79, -1.48, 1.54, 0.59, -0.52, 0.91, 0.75]$$

4. **Data Integrity**:
   - **Projection**: Transforms data to emphasize significant features without altering underlying relationships.
   - **Explained Variance Ratio**: Quantifies the percentage of total variance represented by each principal component.
5. **Practical Visualization**:
   - Compare scatter plots before and after PCA to verify consistency in data trends and clusters.
6. **Applications**:
   - **Face Recognition**: PCA in Eigenfaces shows effective feature retention.
   - **Genomics**: Highlights patterns without distorting genetic information.
7. **Conclusion**:
   - PCA ensures that dimension reduction does not compromise the data's fundamental characteristics, supporting its use in critical data analysis and feature reduction tasks.



Validating PCA's Representation of Original Data

$$Z_1 = [-1.50, -1.85, 0.97, 0.79, -1.48, 1.54, 0.59, -0.52, 0.91, 0.75]$$

